Got it — the core of this project is **deep understanding of what's actually happening in the network traffic**, not just throwing ML at it. ML is just th...

PASTED

how to start with this project give phase and step by step plan

Recalled 2 memories

Phased Plan: PCAP Network Event Detection Engine

Given you know basic C/C++ but haven't done much networking work yet, I'd build this in **Python first** (fast iteration, great packet libraries), then optionally port hot paths to C/C++ later if you want performance work. Here's a realistic phase breakdown:

Phase 0 — Environment & Data (few days)

1. **Install tooling:** `scapy` or `pyshark` (wraps tshark/Wireshark) for packet parsing, Wireshark itself for visual inspection.
2. **Get sample PCAPs:** Wireshark's sample capture repo, or generate your own (run `tcpdump`/Wireshark while doing normal browsing, then deliberately break things — block a port with a firewall rule, kill a DNS server, etc.) so you have labeled "bad" traffic to test against.
3. **Goal check:** by end of this phase you can load a PCAP in Python and print every packet's `timestamp, src_ip, dst_ip, ports, flags, seq, ack, len`.

Phase 1 — Parsing & Stream Reconstruction (1–2 weeks)

1. Write a parser that converts raw packets into a flat structured list (the tuple from the doc).
2. Group packets into **TCP streams** using the 4-tuple (src_ip, src_port, dst_ip, dst_port) — handle both directions of a connection as one stream.
3. Separately bucket **UDP/DNS** traffic by transaction ID.
4. **Milestone:** given any PCAP, you can print "here are all TCP streams and their packet count."

Phase 2 — Rule-Based Event Detectors (2–3 weeks)

Build one detector function per event type, starting with the easiest/most valuable:

1. SYN timeout / connection refused
2. Retransmission (fast vs RTO)
3. Zero window
4. RST mid-stream
5. DNS NXDOMAIN / timeout / retransmission
6. Out-of-order packets

Test each detector against a PCAP where you *know* that failure happened (from Phase 0's deliberate-breakage captures). This is your ground truth validation — no ML needed yet.

Phase 3 — Event Tagging & Storage (1 week)

1. Standardize detector output into an `Event(type, timestamp, src, dst, severity, metadata)` object.
2. Dump events to a simple structure (list of dicts, SQLite, or even a CSV) so you can query/filter later.

Phase 4 — Spatial-Temporal Aggregation (2 weeks)

This is the most interesting analytical part:

1. **Spatial:** group events by dst_ip, by subnet, by src_ip — answer "is this one server, one subnet, or one attacker?"
2. **Temporal:** compute inter-arrival times, sliding-window event rates, detect bursts vs periodic vs escalating patterns.
3. **Correlation:** simple time-window joins (e.g., DNS failure on host X within N seconds of connection failure to X).

Phase 5 — Root Cause Inference (1–2 weeks)

Write rule-based heuristics that take correlated events and output a human-readable diagnosis ("DNS server unreachable", "server overloaded", "possible port scan from X"). This is just if/else logic over your event clusters at first.

Phase 6 — ML Layer (optional, later)

Only after Phases 1–5 work reliably: train a classifier on your rule-labeled events to catch ambiguous/novel cases, or rank likely root causes when multiple events co-occur.

Phase 7 — Polish

CLI tool or simple dashboard (even a basic web UI) that takes a PCAP in and spits out a report of detected events + root cause guesses.

My suggestion on where to actually start today: Phase 0 + the first detector in Phase 1/2 — get `scapy` parsing a PCAP and detecting SYN timeouts. That one detector touches

parsing, stream grouping, and event logic all at once, so it's a good forcing function to get the skeleton of the whole pipeline working before you add the other five detectors.

Want me to help you write the actual Phase 0/1 code to get started?

Create a doc